

目 录

1 第一部分 基于 face-api.js 的浏览器端人脸分析实验	1
1.1 实验背景与目标	1
1.2 项目结构与实验环境	1
1.2.1 项目结构	1
1.2.2 页面结构	2
1.3 核心技术与总体流程	2
1.3.1 face-api.js 模型调用	2
1.3.2 输入、推理与绘制流程	3
1.4 功能模块分析	4
1.4.1 图片表情识别	4
1.4.2 年龄与性别估计	4
1.4.3 摄像头实时追踪	5
1.5 实验结果与分析	6
1.6 存在问题与改进方向	7
1.7 本部分小结	7
2 第二部分 后续内容	8
2.1 待补充内容	8
参考文献	9

1 第一部分 基于 face-api.js 的浏览器端人脸分析实验

1.1 实验背景与目标

人脸分析是计算机视觉在人机交互、身份认证、内容理解和智能终端中的典型应用。传统的人脸分析系统通常把模型部署在服务器端，浏览器或客户端只负责采集图像、上传数据和展示结果。这种方式便于集中管理模型，但也带来了部署复杂、网络依赖较强、交互延迟较高等问题。随着 TensorFlow.js 等前端推理框架的发展，经过训练的深度学习模型可以直接在浏览器中加载与执行，使静态网页也能够完成较完整的人脸检测、关键点定位、表情识别和属性估计任务。

本实验基于 code/face-recognition/ 目录中的纯前端人脸分析项目完成。该项目核心依赖为 face-api.js[1]，页面样式使用 Bootstrap 和 MDB，业务逻辑集中在 dist/ 目录下的 JavaScript 脚本中。实验围绕三个实际页面展开：图片表情识别、图片年龄与性别估计、摄像头实时人脸追踪。通过阅读和运行这些页面，可以观察浏览器端人脸分析系统从模型加载、输入获取、模型推理到结果绘制的完整流程。

本部分实验目标如下：第一，分析静态网页项目中 HTML 页面、业务脚本、模型文件和示例资源之间的组织关系；第二，说明 face-api.js 在图片输入和摄像头视频流输入中的调用流程；第三，分别分析表情识别、年龄与性别估计、实时追踪三个模块的功能实现；第四，总结该类前端人脸分析实验的效果、局限和改进方向。

1.2 项目结构与实验环境

1.2.1 项目结构

该项目没有后端服务和复杂构建入口，整体更接近一个可直接静态托管的演示站点。页面入口位于项目根目录，模型权重位于 models/，示例图片、标注人物图片和视频资源位于 public/，实际被页面引用的业务代码位于 dist/。其中 dist/face-api.min.js 是 face-api.js 的浏览器端构建文件，其他脚本则分别对应不同实验功能。

主要页面、脚本和功能对应关系如表 1.1 所示。

表 1.1 人脸分析实验页面与脚本对应关系

页面文件	业务脚本	主要功能
face_expression_recognition.html	dist/expression_script.js	图片表情识别
age_gender_estimation.html	dist/estimation_script.js	年龄与性别估计
webcam_face_tracking.html	dist/webcam_script.js	摄像头实时追踪

模型目录中包含 Tiny Face Detector、SSD MobileNetV1、68 点人脸关键点、表

情识别、年龄性别估计、人脸识别等预训练模型权重。脚本中的 `MODEL_URL` 均设置为 `"/models"`，人物标注样本路径设置为 `"/public/img/labeled_images"`，说明当前版本已经把模型和示例资源改为从本地仓库加载。与完全依赖远程 GitHub Raw URL 或 CDN 的演示相比，本地资源加载能够减少外部网络波动对实验的影响，也更适合课程报告中的复现实验。

1.2.2 页面结构

三个实验页面都采用相似的界面组织方式：页面顶部为导航栏，中间主体为左右分栏布局。左侧通常放置上传控件、摄像头状态或识别结果卡片，右侧为图像或视频展示区域。页面通过 `script` 标签延迟加载 `dist/face-api.min.js` 和当前功能对应的业务脚本，HTML 本身主要负责结构和交互容器，模型加载、推理和绘制逻辑均由 JavaScript 完成。

这种组织方式的优点是职责清晰。HTML 页面只描述用户界面，CSS 负责统一视觉样式，业务脚本负责把输入数据送入模型并把结果写回页面。由于没有后端服务，实验运行时只需要浏览器能够访问项目静态文件；对于摄像头功能，还需要浏览器支持媒体采集接口并允许页面获取摄像头权限 [2]。

1.3 核心技术与总体流程

1.3.1 face-api.js 模型调用

`face-api.js` 是建立在 `TensorFlow.js` 之上的 JavaScript 人脸分析库，封装了常见的人脸检测、关键点定位、人脸描述子、表情识别以及年龄性别估计接口。项目中的脚本通过 `faceapi.nets.*.loadFromUri(MODEL_URL)` 加载对应网络，并使用 `Promise.all` 并行等待模型加载完成。模型加载完成后，脚本才会注册图片上传监听器或启动摄像头连接流程。

不同实验模块加载的网络略有差异。图片表情识别页面加载 Tiny Face Detector、68 点关键点、人脸描述子、表情识别和 SSD MobileNetV1；年龄与性别估计页面加载 Tiny Face Detector、68 点关键点、人脸描述子、年龄性别估计和 SSD MobileNetV1；摄像头追踪页面加载 Tiny Face Detector、68 点关键点、人脸描述子和表情识别模型。虽然部分页面加载了当前流程中未显式使用的网络，但整体仍体现了人脸分析任务的典型链式调用方式。

从数学表达上看，浏览器端人脸分析可以抽象为从输入图像到一组结构化人脸属性的映射。设输入图像为

$$I \in \mathbf{R}^{H \times W \times 3}, \quad (1.1)$$

其中 H 、 W 分别表示图像高度和宽度，3 表示 RGB 三个颜色通道。人脸检测网络首先在图像上预测候选框集合

$$\mathcal{B} = \{b_i = (x_i, y_i, w_i, h_i, s_i) \mid i = 1, 2, \dots, n\}, \quad (1.2)$$

其中 (x_i, y_i) 表示第 i 个检测框左上角坐标， w_i 和 h_i 表示宽度与高度， s_i 表示该框为人脸的置信度。实际绘制时只保留置信度高于阈值 τ 的检测结果，即

$$\mathcal{B}' = \{b_i \mid s_i \geq \tau, b_i \in \mathcal{B}\}. \quad (1.3)$$

在每个检测框内，关键点定位模型继续估计 68 个面部关键点：

$$\mathcal{L}_i = \{(u_{ij}, v_{ij}) \mid j = 1, 2, \dots, 68\}. \quad (1.4)$$

这些关键点描述了眉毛、眼睛、鼻子、嘴唇和脸部轮廓等局部结构，为后续表情和属性估计提供更细粒度的几何信息。表情识别可以表示为对七类表情的概率分布预测：

$$\mathbf{p}_i = \text{softmax}(\mathbf{z}_i), \quad \hat{c}_i = \arg \max_k p_{ik}, \quad (1.5)$$

其中 $\mathbf{z}_i$ 为模型输出的类别得分， p_{ik} 表示第 i 张人脸属于第 k 类表情的概率， $\hat{c}_i$ 为最终显示的主表情。年龄与性别估计也可写成

$$(\hat{a}_i, \hat{g}_i, q_i) = f_{\text{attr}}(I, b_i, \mathcal{L}_i), \quad (1.6)$$

其中 $\hat{a}_i$ 表示估计年龄， $\hat{g}_i$ 表示性别标签， q_i 表示性别分类置信度。由此可见，项目中的链式 API 调用本质上对应了“检测框定位、关键点建模、属性分类或回归”的多阶段推理过程。

1.3.2 输入、推理与绘制流程

图片类页面的流程可以概括为以下五步。

第一步，监听 `input[type=file]` 的 `change` 事件，获得用户上传的图片文件。第二步，通过 `faceapi.bufferToImage` 将本地文件转换为浏览器中的图片对象，并将图片插入结果展示容器。第三步，使用 `faceapi.createCanvasFromMedia(image)` 创建覆盖画布，再调用 `faceapi.matchDimensions` 使画布尺寸与图片显示尺寸一致。第四步，调用人脸检测接口检测图片中的全部人脸，并根据具体任务继续串联关键点、描

述子、表情或年龄性别估计接口。第五步，将检测结果通过 `faceapi.resizeResults` 映射到展示尺寸，再通过 `faceapi.draw` 系列接口绘制检测框、表情标签或自定义标注，同时把结构化结果渲染到侧边栏。

摄像头页面的流程与图片页面类似，但输入从静态图片变为连续视频帧。脚本先通过浏览器媒体采集接口请求摄像头视频流，并把视频流绑定到页面中的视频元素。当 `video` 触发 `play` 事件后，页面创建覆盖画布并启动定时器，每隔 100 ms 对当前视频帧执行一次人脸检测、关键点定位和表情识别。每轮推理结束后，脚本清空画布并重绘最新结果，从而形成实时追踪效果。

1.4 功能模块分析

1.4.1 图片表情识别

图片表情识别由 `dist/expression_script.js` 实现。页面接收用户上传的 JPG、PNG 等常见图片格式，识别完成后在右侧图片上绘制人脸检测框和表情结果，并在左侧结果区域展示每张人脸的详细表情概率。

脚本预先定义了七类表情标签：中性、高兴、难过、生气、害怕、厌恶、惊讶。这些标签与 `face-api.js` 返回的 `neutral`、`happy`、`sad`、`angry`、`fearful`、`disgusted`、`surprised` 相对应。识别结果返回后，脚本按照检测框的横坐标对多张人脸排序，使页面展示顺序与图像中从左到右的人脸位置一致。随后，脚本遍历七类表情分数，选出概率最高的一类作为主表情，并用百分比和进度条展示每一类表情的模型置信度。

该模块的实现重点不只是调用表情模型，还包括结果解释和用户界面反馈。若没有检测到人脸，页面会显示“未检测到人脸”的空状态提示；若正在处理图片，结果区会先显示“正在识别”。这些状态反馈降低了用户对模型响应时间的困惑，也使实验页面更接近完整应用而非单纯的代码示例。

1.4.2 年龄与性别估计

年龄与性别估计由 `dist/estimation_script.js` 实现。该模块与表情识别共用相似的图片上传和画布绘制流程，但推理链路改为在人脸检测之后继续执行关键点提取、人脸描述子提取以及年龄性别估计。模型返回的主要结果包括检测框、估计年龄、性别标签和性别置信度。

在结果展示上，脚本将年龄四舍五入后以“岁”为单位显示，将 `male` 和 `female` 映射为“男性”和“女性”，并通过结果卡片展示每张人脸的年龄估计、性别判断和性别置信度。同时，脚本使用 `faceapi.draw.DrawBox` 在图片上叠加检测框标签，使用户能够把侧边栏中的结构化数据与图片中的具体人脸对应起来。

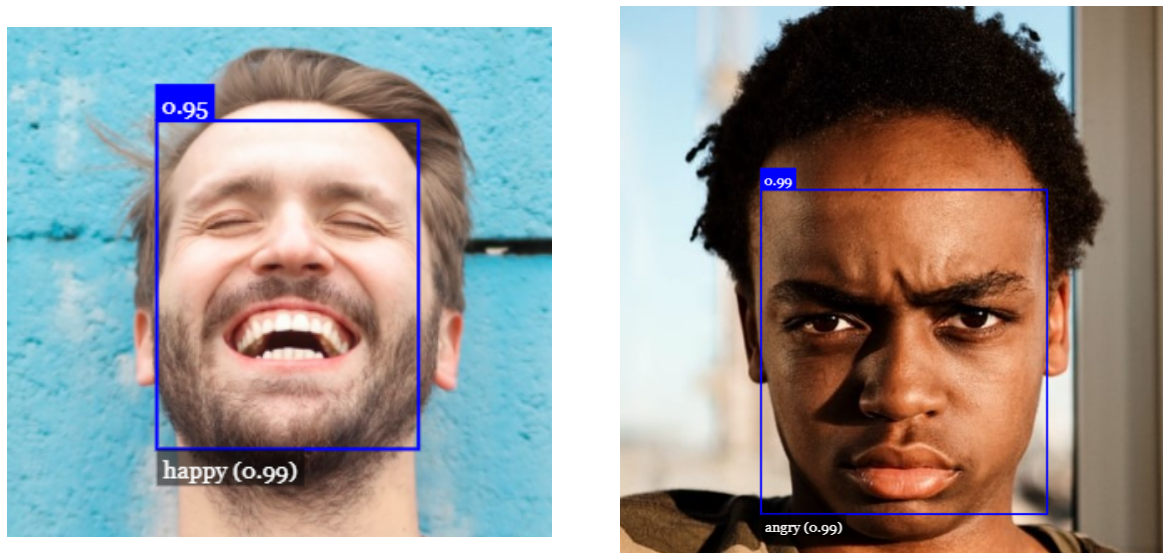


图 1.1 表情识别示例结果

该模块反映了多任务人脸属性估计的基本思想：人脸检测首先确定候选区域，关键点与特征提取提供面部结构信息，属性估计网络再输出年龄和性别等外观属性。需要注意的是，年龄和性别估计受光照、姿态、遮挡、图像清晰度和训练数据分布影响明显，其结果应理解为模型预测值，而不是绝对真实标签。在实际系统中，这类输出尤其不适合直接用于高风险判断。

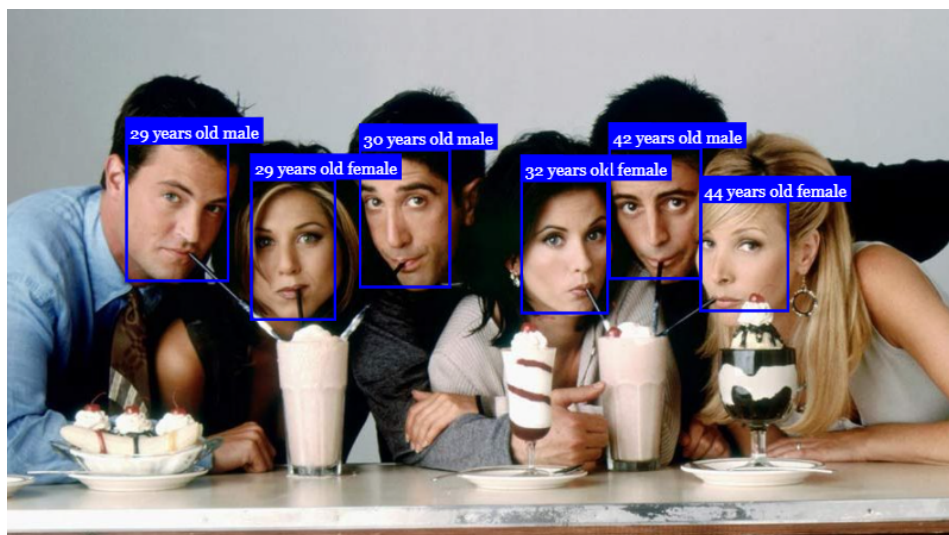


图 1.2 年龄与性别估计示例结果

1.4.3 摄像头实时追踪

摄像头实时追踪由 `dist/webcam_script.js` 实现。页面启动后首先显示“等待授权”状态，随后检查浏览器是否支持 `getUserMedia`。如果浏览器支持该接口，脚本请求摄像头权限；如果用户拒绝授权或设备不可用，页面会显示访问失败提示。成

功获取视频流后，页面等待视频播放事件，并开始周期性识别。

实时识别部分使用 `setInterval` 以 100 ms 为间隔处理视频帧。每一轮识别都会调用 `Tiny Face Detector` 检测人脸，再串联关键点定位和表情识别。脚本随后根据当前展示区域尺寸调整画布，清除上一帧绘制内容，并重新绘制检测框、68 点关键点和表情标签。与图片页面相比，摄像头页面更强调连续性和响应速度，因此选用轻量级检测器更符合实时场景需求。

该模块还处理了若干工程细节。首先，`syncCanvasSize` 会根据展示容器尺寸同步画布大小，窗口尺寸变化时也会重新匹配，避免视频画面与识别标注错位。其次，侧边栏结果设置了 280 ms 的刷新节流，并在结果内容没有变化时跳过重复渲染，降低高频检测对 DOM 更新的压力。最后，脚本监听视频暂停和结束事件，及时清除定时器并更新页面状态，避免后台持续执行无意义的推理任务。

1.5 实验结果与分析

从功能完整性看，实验系统完成了三个典型的人脸分析任务。表情识别页面能够在上传图片后检测多张人脸，并输出每张人脸的主表情及七类表情概率；年龄与性别估计页面能够输出检测框、估计年龄、性别标签和置信度；摄像头追踪页面能够在获得权限后持续处理视频帧，并实时叠加检测框、关键点和表情结果。

从工程实现看，该项目具有较好的演示性。项目采用静态资源组织方式，页面入口清晰，业务脚本集中，模型文件和示例数据保存在本地目录，便于离线讲解和复现实验。左侧结果区与右侧图像区域的布局也有利于对照观察模型输出：右侧用于判断人脸定位和标注位置是否正确，左侧用于查看更细粒度的概率、置信度和状态信息。

从性能角度看，图片类任务主要受模型加载时间和单次图片推理时间影响；摄像头任务还受到设备计算能力、摄像头分辨率、浏览器性能和当前页面负载影响。脚本采用 `Tiny Face Detector` 进行实时检测，并对侧边栏渲染做节流处理，说明项目已经考虑到实时场景中的性能压力。不过，如果输入视频分辨率较高或设备性能较弱，实时帧率仍可能下降。

从可靠性角度看，该系统的结果受数据质量影响较大。正脸、光照均匀、遮挡较少的人像通常更容易得到稳定结果；侧脸、模糊、强背光、多人脸重叠或表情不明显的图片可能导致漏检、误检或属性估计偏差。摄像头功能还依赖浏览器权限和运行上下文，某些浏览器或非安全环境可能无法直接访问摄像头。因此，本实验更适合作为前端深度学习推理和人脸分析流程的教学演示，而不应被理解为生产级身

份识别系统。

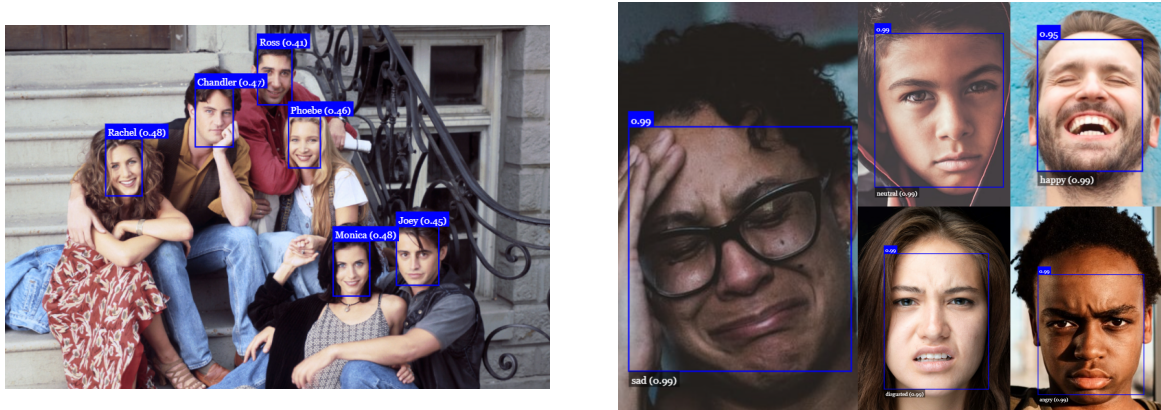


图 1.3 人脸检测与综合演示效果

1.6 存在问题与改进方向

当前项目已经能够展示浏览器端人脸分析的基本能力，但仍存在可以改进的地方。

第一，页面和脚本之间仍以全局 DOM 查询和普通脚本文件组织为主，随着功能增多，状态管理和代码复用会变得困难。后续可以把公共的模型加载、画布匹配、空状态渲染、结果排序等逻辑抽取为共享模块。

第二，当前实验主要依赖预训练模型的直接输出，缺少定量评估。若要把实验写得更严谨，可以构造包含不同光照、姿态、遮挡和多人脸场景的测试集，统计检测成功率、属性估计稳定性和实时帧率等指标。

第三，摄像头功能涉及人脸和视频流等敏感信息。虽然当前项目在浏览器端本地处理数据，没有主动上传视频，但报告和界面仍应明确提示用户授权含义，并避免保存或传播未经同意的人脸图像。若继续扩展为实际应用，还需要加入更完整的隐私保护、权限说明和数据生命周期管理。

第四，年龄和性别估计属于外观属性推断，天然存在不确定性和潜在偏差。后续可以在界面和报告中弱化确定性表达，例如将结果称为“模型估计”而非“真实年龄”或“真实性别”，并在实验分析中说明其不适用于身份、资格、风险等高影响决策。

1.7 本部分小结

本部分围绕 face-recognition 项目完成了浏览器端人脸分析实验说明。实验展示了静态网页如何利用 face-api.js 加载本地模型，并在图片和摄像头视频流两类输入上完成人脸检测、关键点定位、表情识别以及年龄性别估计。通过分析三个

页面的结构和脚本流程可以看出，前端深度学习推理适合课程演示、交互原型和轻量级体验场景，但在性能、鲁棒性、隐私保护和结果可信度方面仍需要进一步评估与完善。

2 第二部分 后续内容

本报告第二部分用于补充其他实验或扩展分析内容。当前先保留章节结构，后续可根据课程要求继续撰写研究背景、方法设计、实验过程、结果分析和改进方向。

2.1 待补充内容

本节暂不展开。后续写作时，建议与第一部分保持一致的结构：先说明问题背景和目标，再介绍系统结构或算法流程，最后给出实验结果、局限性和改进方案。

参考文献

- [1] Vincent Muhler. face-api.js: JavaScript API for face detection and face recognition in the browser and nodejs with tensorflow.js. GitHub, 2018.
- [2] World Wide Web Consortium. Media Capture and Streams. W3C Recommendation.
- [3] Bootstrap Team. Bootstrap Documentation.